- Ejercicio ciudades de Canadá -

```
% Usar la libreria de listas
:- use_module(library(lists)).

% Conexiones dirigidas entre ciudades y sus costos
conexion(vancouver, edmonton, 16).
conexion(vancouver, calgary, 13).
conexion(calgary, edmonton, 4).
conexion(edmonton, saskatoon, 12).
conexion(calgary, regina, 14).
conexion(regina, saskatoon, 7).
conexion(saskatoon, calgary, 9).
conexion(saskatoon, winnipeg, 20).
conexion(regina, winnipeg, 4).

% 1. Consultar si un nodo tiene aristas salientes
tiene_aristas(X) :-
    conexion(X, _, _).

% 2. Consultar las conexiones directas y sus costos
conectado_a(X, Y, Costo) :-
    conexion(X, Y, Costo).

% 3. Calcular el costo de ir de X a Z pasando por Y
costo_por(X, Y, Z, CostoTotal) :-
    conexion(X, Y, Costo1),
    conexion(Y, Z, Costo2),
    CostoTotal is Costo1 + Costo2.

% 4. Comprobar si existe una ruta entre dos ciudades
ruta(X, Y) :-
    buscar_ruta(X, Y, [X]).

buscar_ruta(X, Y, _) :-
    conexion(X, Y, _).

buscar_ruta(X, Y, Visitadas) :-
    conexion(X, Z, _),
    \+ member(Z, Visitadas),
    buscar_ruta(Z, Y, [Z|Visitadas]).
```

- Simpson -

```
% Libreria de listas
:- use_module(library(lists)).

% Hechos: padres y madres
padre(abraham, herbert).
```

```
padre(abraham, homero).

madre(mona, herbert).
madre(mona, homero).

padre(clancy, marge).
padre(clancy, selma).
padre(clancy, patty).

madre(jacqueline, marge).
madre(jacqueline, selma).
madre(jacqueline, patty).

padre(homero, bart).
padre(homero, lisa).
padre(homero, maggie).

madre(marge, bart).
madre(marge, lisa).
madre(marge, maggie).

% Lista de hijos de cada pareja de hermanos
hijos(herbert, []).
hijos(homero, [bart, lisa, maggie]).
hijos(marge, [bart, lisa, maggie]).

% Regla para identificar hermanos
hermanos(X, Y) :-
    padre(P, X),
    padre(P, Y),
    X \= Y.

hermanos(X, Y) :-
    madre(M, X),
    madre(M, Y),
    X \= Y.

% Regla para identificar abuelos
abuelo(X, Y) :-
    padre(X, Z),
    padre(Z, Y).

abuelo(X, Y) :-
    padre(X, Z),
    madre(Z, Y).

abuela(X, Y) :-
    madre(X, Z),
```

```
    padre(Z, Y).

abuela(X, Y) :-
    madre(X, Z),
    madre(Z, Y).

% Regla para identificar tios
tio(X, Y) :-
    hermanos(X, Z),
    padre(Z, Y).

tio(X, Y) :-
    hermanos(X, Z),
    madre(Z, Y).

% Regla para identificar hijos usando una lista
hijo_de(X, Y) :-
    hijos(Y, Lista),
    member(X, Lista).
```